Supplementary Materials for

EMOJI: Extension and Modification of AdaBelief for Adaptive Step Adjustment and Improved Robustness

Mojtaba Pourtahmasbi
Department of Computer Engineering and Information Technology, La.C., Islamic Azad University,
Lahijan, Iran
Mojtaba.Pourtahmasbi@iau.ir
https://orcid.org/0009-0002-4376-2868

Kamrad Khoshhal Roudposhti
Department of Computer Engineering and Information Technology, La.C., Islamic Azad University,
Lahijan, Iran
Kamrad@liau.ac.ir
https://orcid.org/0000-0001-9633-3114

Mehraneh Pourtahmasbi
Department of Computer Engineering and Information Technology, Ra.C., Islamic Azad University,
Rasht, Iran
mehraneh.pourtahmsbi@iau.ir
https://orcid.org/0009-0001-1570-5834

Mahmoud Alimoradi
Department of Computer Engineering and Information Technology, La.C., Islamic Azad University,
Lahijan, Iran
Alimoradi@iau.ac.ir
https://orcid.org/0000-0001-6167-439X

This supplementary material provides additional experimental details, supplementary tables and figures, ablation studies, sensitivity analyses, computational results, and other supporting information. To ensure reproducibility, the complete source code, per-seed evaluations, and experimental configurations are publicly available at https://github.com/mojipoori/emoji_optimizer.

1. Theoretical Analysis of Adaptive Denominator Dynamics

We analyze the adaptive denominator from two perspectives: the effect of the stability constant ϵ on the recursive second-moment state, and the potential benefit of estimating the second central moment of the gradient prediction error.

1.1. Assumption

To analyze the effect of this mechanism in a regime where the second-moment state has locally stabilized, we assume that the statistical dynamics of the gradients evolve sufficiently slowly during training such that the second-moment state approaches a quasi-steady regime. Under this assumption, the expected values of consecutive states can be approximated as

$$\mathbb{E}[s_t] \approx \mathbb{E}[s_{t-1}] = \bar{s}.$$

This assumption does not imply that the gradients themselves remain stationary throughout training. Rather, it provides a local steady-state approximation for analyzing the behavior of the second-moment accumulator.

1.2. Stability Constant in the Recursive Second-Moment State

We first examine how the implementation of the stability constant ϵ can affect the adaptive denominator in AdaBelief.

In the official AdaBelief implementation, the second-moment state is first updated according to

$$s_t \leftarrow \beta_2 s_{t-1} + (1-\beta_2) d_t^2.$$

after which the stability constant is added directly to the state,

$$s_t \leftarrow s_t + \epsilon.$$

The resulting effective recurrence can therefore be written as

$$s_t = \beta_2 s_{t-1} + (1-\beta_2) d_t^2 + \epsilon.$$

Taking expectations of both sides gives

$$\mathbb{E}[s_t] = \beta_2 \mathbb{E}[s_{t-1}] + (1-\beta_2)\mathbb{E}[d_t^2] + \epsilon.$$

Under the quasi-steady-state assumption,

$$\mathbb{E}[s_t] \approx \mathbb{E}[s_{t-1}] = \bar{s},$$

which gives

$$\bar{s} = \beta_2 \bar{s} + (1-\beta_2)\mathbb{E}[d_t^2] + \epsilon.$$

Rearranging the terms yields

$$(1-\beta_2)\bar{s} = (1-\beta_2)\mathbb{E}[d_t^2] + \epsilon.$$

and therefore

$$\boxed{\bar{s} = \mathbb{E}[d_t^2] + \frac{\epsilon}{1-\beta_2}}.$$

The equation shows that incorporating ϵ into the recursive second-moment state introduces a persistent contribution of $\epsilon/(1-\beta^2)$ to its steady-state value. This contribution is independent of the observed magnitude of the gradient prediction error and therefore remains present even when the underlying gradient statistics become small.

Consequently, when the underlying second-moment statistic becomes small, the relative contribution of $\epsilon/(1-\beta^2)$ becomes more pronounced. The recursive inclusion of ϵ can therefore make the adaptive denominator larger than it would be without this contribution and can limit its reduction relative to the underlying second-moment statistic. Assuming a fixed numerator and learning rate, this can impose additional damping on the effective adaptive step.

$$denom_{AdaBelief} = \sqrt{\mathbb{E}[d_t^2] + \frac{\epsilon}{1-\beta_2}} + \epsilon.$$

The key issue is therefore not the use of ϵ itself, which is necessary for numerical stability, but its inclusion within the recursive second-moment state.

In EMOJI[1], ϵ is removed from the recursive second-moment state and applied only after the square-root operation.

$$denom_{EMOJI} = \sqrt{\mathbb{E}[d_t^2]} + \epsilon,$$

where $denom_{EMOJI}$ denotes the modified second-moment statistic. This formulation preserves the numerical role of ϵ while preventing it from becoming part of the recursively accumulated state.

This analysis motivates the first modification in EMOJI: removing ϵ from the recursive second-moment state while retaining it in the final denominator.

[1] **E**xtension and **Mo**dification of AdaBelief for Adaptive Step Ad**j**ustment and **I**mproved Robustness

1.3. Estimating the Second Central Moment of the Gradient Prediction Error

As an auxiliary mechanism, we consider whether estimating the second central moment of the gradient prediction error can reduce the magnitude of the second-moment statistic underlying the adaptive denominator.

In AdaBelief, the gradient prediction error is defined as

$$d_t = g_t - m_t,$$

where g_t is the current gradient and m_t is its exponential moving average. The second-moment state is then updated as

$$s_t = \beta_2 s_{t-1} + (1-\beta_2) d_t^2.$$

Under the quasi-stationary assumption, the quantity underlying this update corresponds to the raw second moment of d_t, $\mathbb{E}[{d_t}^2]$. The raw second moment can be decomposed as

$$\mathbb{E}[d_t^2] = Var(d_t) + (\mathbb{E}[d_t])^2.$$

Therefore, when the mean of d_t is not exactly zero, the raw second moment contains an additional non-negative component corresponding to the squared mean, in addition to the second central moment, or variance of d_t.

To account for this possibility, EMOJI maintains an additional exponential moving average of d_t:

$$v_t = \beta_1 v_{t-1} + (1-\beta_1) d_t.$$

The second moment state is then computed as follows:

$$j_t = \beta_2 j_{t-1} + (1-\beta_2)(d_t - v_t)^2.$$

Under the quasi-stationary assumption, when v_t adequately tracks the local mean of d_t, we have

$$v_t \approx \mathbb{E}[d_t],$$

and consequently,

$$\mathbb{E}[(d_t - v_t)^2] \approx Var(d_t).$$

Under the same local quasi-stationary approximation, the corresponding steady-state second-moment quantities therefore satisfy

$$\sqrt{Var(d_t) + (\mathbb{E}[d_t])^2} \geq \sqrt{Var(d_t)}\,.$$

Consequently,

$$\sqrt{s_t} \geq \sqrt{j_t}\,,$$

when a non-zero local mean component is present, the centered formulation can yield a smaller second-moment quantity by removing the contribution associated with $(\mathbb{E}[d_t])^2$. At the level of local statistical behavior, this can result in a smaller adaptive denominator.

1.4. Computational Complexity and Memory

EMOJI preserves the element-wise structure of AdaBelief and therefore has the same asymptotic computational and memory complexity, namely $O(N)$ per optimization step for N trainable parameters. The centering mechanism introduces one additional parameter-sized moving-average state, but does not change the asymptotic complexity of the optimizer.

In our experiments, EMOJI and AdaBelief showed comparable peak GPU memory usage and similar practical training time. Thus, although EMOJI maintains an additional optimizer state, our experiments did not show a substantial increase in overall GPU memory consumption or training time under the evaluated configurations.

2. Experimental Setup

Hyperparameter selection can substantially affect convergence speed, training stability, and the final generalization performance of deep learning models. In comparative studies of different optimizers, inappropriate or inconsistent hyperparameter choices may introduce unintended biases and complicate the interpretation of the results. Therefore, particular care was taken to establish a consistent and reproducible experimental protocol across the evaluated models and tasks.

Whenever possible, hyperparameter settings were adopted from the corresponding reference papers and established configurations for the evaluated optimizers and model architectures, including ResNet, DenseNet, Transformer, GPT-style Transformer, and ALBERT. Optimizer-specific parameters were selected according to their respective reference configurations or the experimental procedures described in the following sections.

To account for variability arising from stochastic training, experiments were repeated using multiple random seeds whenever computational resources permitted. For each experiment involving multiple seeds, we report the mean and standard deviation of the resulting performance measurements. The same set of random seeds was used for the optimizers compared within each experimental configuration. This shared-seed protocol was intended to reduce the influence of seed-dependent factors and provide a more controlled comparison of optimizer performance. Relevant sources of randomness include weight initialization, training-data shuffling and mini-batch composition, stochastic data augmentation, and dropout. Although using common seeds does not eliminate every source of randomness, it provides a consistent basis for comparing the evaluated methods.

All experiments were conducted using an NVIDIA GeForce RTX 3070 Ti GPU with 8 GB of VRAM. In a small number of experiments, minor adjustments to the batch size were necessary because of GPU memory constraints; these changes are explicitly reported in the corresponding experimental configurations.

To ensure a consistent comparison among optimizers, we applied the same weight decay formulation to all optimizers within each experiment. Depending on the experimental setting, we used either L2 regularization (coupled weight decay) or decoupled weight decay [1], allowing us to compare the optimizers under a common regularization protocol. Note that, in our experiments, SGD denotes stochastic gradient descent with momentum [2], whereas SGDW denotes stochastic gradient descent with momentum and decoupled weight decay [1]. The following sections provide the hyperparameter configurations used for each dataset and model architecture, including learning rates, optimizer-specific parameters, weight decay, learning-rate schedules, warmup strategies, and other relevant training settings.

Table S 1: Weight decay formulations used across benchmarks for all optimizers.[1]

Benchmark	CIFAR-10	WikiText-2	IWSLT14	CoLA	MRPC	STS-B	ImageNet-1K
WD Type	Decoupled	Decoupled	Decoupled	Decoupled	Decoupled	Decoupled	Decoupled

2.1. Epsilon Selection and Fixed-Hyperparameter Evaluation

To evaluate the robustness and scalability of the optimizers across different experimental scenarios under a common protocol, we fixed the numerical-stability constant at $\epsilon = 10^{-8}$ for all Adam [3], AdamW [1], AdaBelief [4], and EMOJI experiments throughout the main experiments. For AdaBelief, this choice is consistent with the configuration reported in the original AdaBelief paper, where $\epsilon = 10^{-8}$ was used for the reported CIFAR and ImageNet experiments [4].

Importantly, the reported settings in the original paper differ in some cases from the later task-dependent recommendations provided in the official AdaBelief repository. The repository reports different recommended ϵ values for different tasks, including smaller values for Transformer-based models, and also notes that ϵ should be tuned according to the task.

[1] **Abbreviations:** WD, weight decay.

We therefore do not selectively apply these later task-dependent recommendations to AdaBelief in the main experiments. Doing so would introduce task-specific hyperparameter tuning for the baseline and would confound the fixed-hyperparameter comparison. Instead, we retain $\epsilon = 10^{-8}$ for AdaBelief throughout the primary experiments and investigate the effect of alternative ϵ values separately in the sensitivity analysis. This fixed-ϵ protocol is not intended to identify the individually optimal ϵ for each optimizer and task. Rather, it provides a common evaluation setting for examining whether the optimizers can maintain competitive and consistent performance across different architectures and tasks without task-specific ϵ tuning.

The epsilon values for Yogi [5] and AdaBound [6] were selected based on their respective reference implementations and publications. For Yogi, we used $\epsilon = 10^{-3}$, consistent with the original paper, which explicitly adopts this value across its experiments to control the optimizer's adaptivity [5]. For AdaBound, we used $\epsilon = 10^{-8}$, matching the default value specified in the authors' official implementation [6]. For Adam [3] and AdamW [1], we used $\epsilon = 10^{-8}$, consistent with the default value specified in the official PyTorch implementations.

Table S 2: ϵ values used for adaptive optimizers across experiments.

Optimizer	Adam	AdamW	AdaBelief	Yogi	AdaBound	EMOJI
ϵ	10^{-8}	10^{-8}	10^{-8}	10^{-3}	10^{-8}	10^{-8}

2.2. CIFAR-10 Image Classification with DenseNet121, ResNet20, and ResNet34

Experiments on CIFAR-10 used the standard training and test splits, consisting of 50,000 and 10,000 RGB images, respectively, with a spatial resolution of 32×32 pixels. During training, data augmentation consisted of random cropping to 32×32 pixels after padding by four pixels, followed by random horizontal flipping. Images were then converted to tensors and normalized using the channel-wise means of (0.4914, 0.4822, 0.4465) and standard deviations of (0.2023, 0.1994, 0.2010). For evaluation, test images were converted to tensors and normalized using the same statistics, without data augmentation. The default batch size was 128, and cross-entropy loss was used as the classification objective. Optimizer settings used in each experiment are reported in the corresponding architecture-specific subsections and tables.

Table S 3: Summary of the CIFAR-10 model architectures used in the experiments.

Setting	ResNet-20	ResNet-34	DenseNet-121
Input channels	3	3	3
Input resolution	32 × 32 expected	32 × 32 expected	32 × 32 expected
Stem convolution	3 × 3, 16 channels	3 × 3, 64 channels	3 × 3, 24 channels
Number of downsampling	2	3	3
Stage/block configuration	[3, 3, 3]	[3, 4, 6, 3]	[6, 12, 24, 16]
Growth rate	N/A	N/A	12
Number of classes	10	10	10
Normalization	BatchNorm2d	BatchNorm2d	BatchNorm2d
Activation	ReLU	ReLU	ReLU
Batch size	128	128	128

2.2.1. Image Classification with ResNet-20

ResNet-20 was implemented using a three-stage residual architecture with BasicBlock residual units in `pytorch`. The initial convolution uses a 3×3 kernel with 16 output channels, followed by three residual stages containing three blocks each. The channel widths of these stages are 16, 32, and 64, respectively. Spatial

downsampling is performed at the beginning of the second and third stages using a stride of two. An 8×8 average-pooling operation reduces the final feature maps to a single spatial value per channel, followed by a fully connected layer with ten output units.

To evaluate the proposed EMOJI optimizer against AdamW, AdaBelief, Yogi, AdaBound, Lion, and SGDW, we conducted experiments on CIFAR-10 using the ResNet-20 architecture. All optimizers were evaluated under the same hardware and software conditions. The model was trained for 180 epochs with a batch size of 128, and each experiment was repeated using four independent random seeds. The mean performance across the repeated runs was used as the primary evaluation criterion.

For all optimizers, a common search range was considered for the learning rate, and the selected values were determined based on the validation performance obtained during the hyperparameter search. The parameter β_2 was similarly evaluated over the specified search range for optimizers with a corresponding parameter. The search ranges and selected values are summarized in Table S 4.

Weight decay was implemented in a decoupled form for all evaluated optimizers. A value of 5×10^{-4} was used for AdamW, AdaBelief, Yogi, AdaBound, EMOJI, and SGDW, whereas the weight decay value for Lion was configured according to recommendations in the Lion literature [7]. Its weight decay was increased tenfold, and set to 5×10^{-3}. This adjustment is motivated by Lion's sign-based update, which tends to produce updates with larger norms than AdamW's and therefore commonly calls for a smaller learning rate and larger weight-decay coefficient.

To improve training stability, a linear warmup schedule was applied at the beginning of training for all optimizers. During the warmup phase, the learning rate was increased linearly from a very small initial value to the selected learning rate over 250 optimization steps. After warmup, a step-wise learning-rate scheduler was applied at epoch 110, reducing the learning rate by a factor of 10 for all evaluated methods.

For the numerical-stability constant, $\epsilon = 10^{-8}$ was used for AdamW, AdaBelief, AdaBound, and EMOJI. For Yogi, $\epsilon = 10^{-3}$ was used according to the reference configuration, and its initial second-moment accumulator was set to 10^{-6}. For AdaBound, the optimizer-specific `final_lr` and γ parameters were set to 0.1 and 10^{-3}, respectively. The complete hyperparameter search ranges and selected configurations are summarized in Table S 4.

Table S 4: Hyperparameter search ranges and selected configurations of the evaluated optimizers on ResNet-20 with CIFAR-10.[1]

Optimizer	LR Search Range	Selected LR	β_1	β_2 Search Range	Selected β_2	WD Type	WD Value	ϵ
AdamW	{0.1,0.01,0.001,0.0001}	0.01	0.9	{0.99, 0.999}	0.99	Decoupled	5×10^{-4}	10^{-8}
AdaBelief	{0.1,0.01,0.001,0.0001}	0.01	0.9	{0.99, 0.999}	0.99	Decoupled	5×10^{-4}	10^{-8}
Yogi	{0.1,0.01,0.001,0.0001}	0.01	0.9	{0.99, 0.999}	0.99	Decoupled	5×10^{-4}	10^{-3}
EMOJI (Proposed)	{0.1,0.01,0.001,0.0001}	0.01	0.9	{0.99, 0.999}	0.99	Decoupled	5×10^{-4}	10^{-8}
AdaBound	{0.1,0.01,0.001,0.0001}	0.001	0.9	{0.99, 0.999}	0.99	Decoupled	5×10^{-4}	10^{-8}
Lion	{0.1,0.01,0.001,0.0001}	0.001	0.9	{0.99, 0.999}	0.99	Decoupled	5×10^{-3}	—
SGDW	{0.1,0.01,0.001,0.0001}	0.1	0.9	—	—	Decoupled	5×10^{-4}	—

[1] **Abbreviations**: LR denotes learning rate, and WD denotes weight decay. For SGDW, the coefficient reported under β_1 is the momentum coefficient. For Lion, β_2 is an optimizer-specific smoothing coefficient rather than a second-moment parameter. For SGDW, the β_1 entry denotes the momentum coefficient rather than an exponential first-moment coefficient.

2.2.2. Image Classification with ResNet-34 and DenseNet-121

The CIFAR-10 implementation of ResNet-34 uses the four-stage BasicBlock configuration [3, 4, 6, 3]. Unlike the conventional ImageNet-oriented stem, it begins with a 3×3 convolution with stride one and 64 output channels, without an initial max-pooling operation. The four residual stages use 64, 128, 256, and 512 channels, with spatial downsampling at the beginning of the last three stages. A 4×4 average-pooling operation is applied before the final fully connected classification layer.

DenseNet-121 was implemented using a CIFAR-10-specific configuration with four dense blocks containing [6, 12, 24, 16] bottleneck layers. The initial convolution produces 24 feature channels, and the growth rate is set to 12. Each bottleneck layer applies batch normalization and ReLU activation before its convolutional operations, and its newly generated feature maps are concatenated with the block input. Three transition layers connect the four dense blocks; each transition applies batch normalization, ReLU activation, a 1×1 convolution, and 2×2 average pooling. A final batch-normalization and ReLU operation, followed by average pooling and a fully connected layer, produces the ten-class output.

ResNet-34 was trained for 200 epochs, while DenseNet-121 was trained for 180 epochs. All experiments were performed with a batch size of 128 and repeated using four independent random seeds. The mean performance across the repeated runs was used as the primary evaluation criterion.

To maintain a consistent comparison while accounting for the different network architectures, the same optimizer configurations were used for both models. Rather than performing an extensive hyperparameter search for each optimizer and architecture, we adopted configurations based on established settings reported in the literature and the corresponding reference implementations. The optimizer configurations are summarized in Table S 5.

Table S 5: Optimizer hyperparameter configurations used for ResNet-34 and DenseNet-121 on CIFAR-10.

Optimizer	**LR**	$\boldsymbol{\beta_1}$	$\boldsymbol{\beta_2}$	**WD Type**	**WD Value**	$\boldsymbol{\epsilon}$
AdamW	0.001	0.9	0.999	Decoupled	0.01	10^{-8}
AdaBelief	0.001	0.9	0.999	Decoupled	0.01	10^{-8}
Yogi	0.001	0.9	0.999	Decoupled	0.01	10^{-3}
EMOJI (Proposed)	0.001	0.9	0.999	Decoupled	0.01	10^{-8}
AdaBound	0.001	0.9	0.999	Decoupled	0.01	10^{-8}
Lion	0.0001	0.9	0.99	Decoupled	0.1	—
SGDW	0.1	0.9	—	Decoupled	0.01	—

For all optimizers, a linear warmup schedule was applied during the first 750 optimization steps. The learning rate was increased linearly during warmup until reaching the corresponding value specified in Table S 5. After warmup, training continued using the selected learning rate. At epoch 140, a step-wise learning-rate scheduler reduced the learning rate by a factor of 10 for all optimizers, after which training continued with the reduced learning rates until the end of the respective training schedules.

Weight decay was implemented in a decoupled form for all evaluated optimizers. A value of 0.01 was used for AdamW, AdaBelief, Yogi, EMOJI, AdaBound, and SGDW, whereas the weight decay value for Lion was configured according to recommendations in the Lion literature. Its weight decay was increased tenfold, and set to 0.1. This adjustment is motivated by Lion's sign-based update, which tends to produce updates with larger norms than AdamW's and therefore commonly calls for a smaller learning rate and a larger weight-decay coefficient. For all optimizers except SGDW and Lion, the learning rate was set to 0.001. As mentioned, Lion typically requires a smaller learning rate, as noted in its original paper. Accordingly, its learning rate was set to one-tenth of AdamW's [7].

For Yogi, the initial second-moment accumulator was set to 10^{-6}, following the configuration used in the corresponding reference. For AdaBound, the optimizer-specific `final_lr` and γ parameters were set to 0.1 and 10^{-3}, respectively. The numerical-stability constants followed the protocol described in Section S 2.1.

2.3. GPT-style Transformer Language Modeling on WikiText-2

We evaluated four optimization algorithms—AdamW, AdaBelief, EMOJI, and Lion—on autoregressive language modeling using WikiText-2. All experiments used the same GPT-style Transformer architecture, dataset splits, training configuration, and set of three shared random seeds to support a consistent comparison across optimizers.

2.3.1. Model architecture and training configuration

The model was implemented using Fairseq's `transformer_lm_gpt` architecture. It comprised four Transformer decoder layers, an embedding dimension of 128, four attention heads per layer, and a feed-forward network dimension of 512. The model used GELU activation, dropout of 0.1, attention dropout of 0.1, shared input and output embeddings, and a maximum sequence length of 128 tokens.

Table S 6: Model architecture and configuration.

Component	Configuration
Task	Autoregressive language modeling
Dataset	WikiText-2
Fairseq architecture	`transformer_lm_gpt`
Transformer decoder layers	4
Embedding dimension	128
Attention heads per layer	4
Feed-forward dimension	512
Attention head dimension	32
Activation function	GELU
Input/output embeddings	Shared
Maximum sequence length	128 tokens
Model dropout	0.1
Attention dropout	0.1

Table S 7: Shared training configuration.

Training parameter	Value
Maximum epochs	50
Maximum tokens per batch	1,024
Gradient accumulation (`update_freq`)	4
Learning-rate scheduler	Inverse square root
Warm-up updates	1,000
Warm-up initial learning rate	10^{-7}
Gradient clipping (`clip_norm`)	0.0 (disabled)
Logging interval	100 updates
Data-loader workers	0
Random seeds	Three shared seeds

The inverse-square-root schedule increased the learning rate during warm-up and subsequently decayed it according to the inverse square root of the update count. All optimizers used the same training schedule and model configuration, while their optimizer-specific hyperparameters were set as described below.

2.3.2. Optimizer configurations

AdamW, AdaBelief, and Emoji were evaluated using the same learning rate of 5×10^{-4}, weight decay of 0.01, and moment coefficients $\beta_1 = 0.9$ and $\beta_2 = 0.98$. The choice of these coefficients follows the original Transformer paper, *Attention Is All You Need* [8], which used Adam with $\beta_1 = 0.9$, $\beta_2 = 0.98$. The present experiments retain the same beta coefficients and a common inverse-square-root learning-rate schedule with 1,000 warm-up updates. The numerical-stability constants followed the protocol described in Section S 2.1. Applying the same settings to AdamW, AdaBelief, and Emoji provides a controlled comparison of their optimization behavior under a shared configuration.

Table S 8: Optimizer configurations used for GPT-style Transformer Language Modeling on WikiText-2.

Optimizer	Learning rate	β_1	β_2	ϵ	Weight decay
AdamW	5×10^{-4}	0.9	0.98	10^{-8}	0.01
AdaBelief	5×10^{-4}	0.9	0.98	10^{-8}	0.01
Emoji	5×10^{-4}	0.9	0.98	10^{-8}	0.01
Lion	5×10^{-5}	0.95	0.98	Not required	0.1

Lion was configured according to recommendations in the Lion literature. Its learning rate was set to one-tenth of the AdamW learning rate, while its weight decay was increased tenfold. This adjustment is motivated by Lion's sign-based update, which tends to produce updates with larger norms than AdamW's and therefore commonly calls for a smaller learning rate and larger weight-decay coefficient [7]. The coefficients $\beta_1 = 0.95$ and $\beta_2 = 0.98$ are also consistent with settings investigated for Lion in language-modeling experiments [7]. These values are optimizer-specific recommendations rather than the default beta values of Lion.

The Lion configuration therefore follows the recommended relative scaling of learning rate and weight decay, rather than enforcing identical hyperparameters across all optimizers. Consequently, the comparison evaluates the four methods under the specified configurations, not under a strict requirement that every optimizer use identical optimizer-specific hyperparameters.

2.3.3. Reproducibility

Each optimizer was evaluated using the same three random seeds, with all other shared model and training settings held constant. This shared-seed protocol helps control for variability due to random initialization and training stochasticity when comparing results across optimizers.

2.3.4. Per-Seed Evaluation

Figure S 1, Figure S 2, and Figure S 3 present the validation loss and validation perplexity over 50 epochs across three different random seeds (1402, 2025, and 1993). Across all seeds and both metrics, the proposed Emoji ($\alpha = 1$) optimizer consistently demonstrates superior performance, achieving the lowest validation loss and perplexity. The Emoji variant without centering ($\alpha = 0$) performs as a close second, indicating the effectiveness of the centering mechanism. Both variants consistently outperform the standard AdamW optimizer. Furthermore, the Lion optimizer exhibits slower convergence, while AdaBelief shows significantly worse performance and fails to converge to competitive levels within the given epochs.

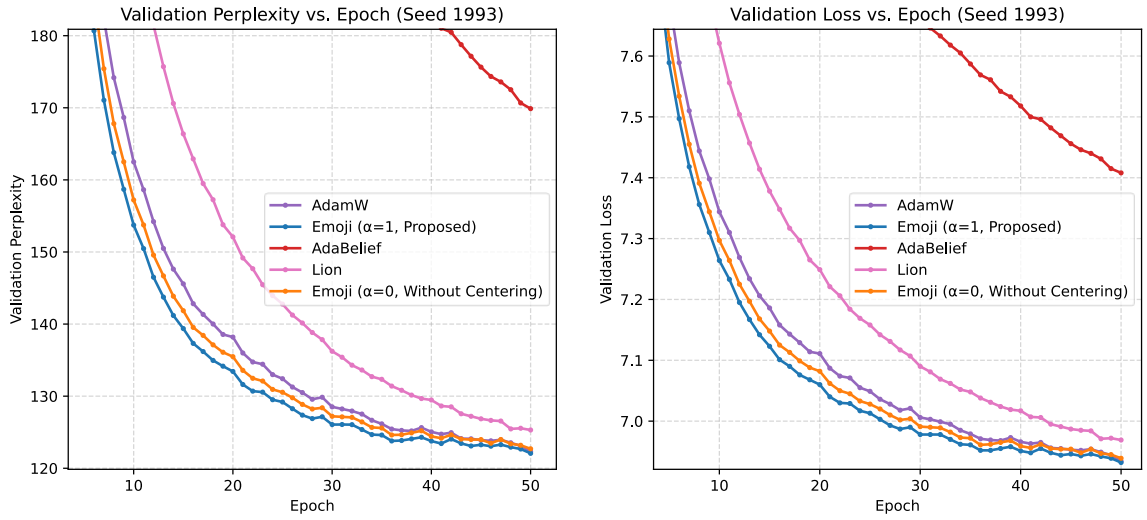


Figure S 1: Validation Perplexity (PPL) and Validation Loss versus Epoch for random seed 1993.

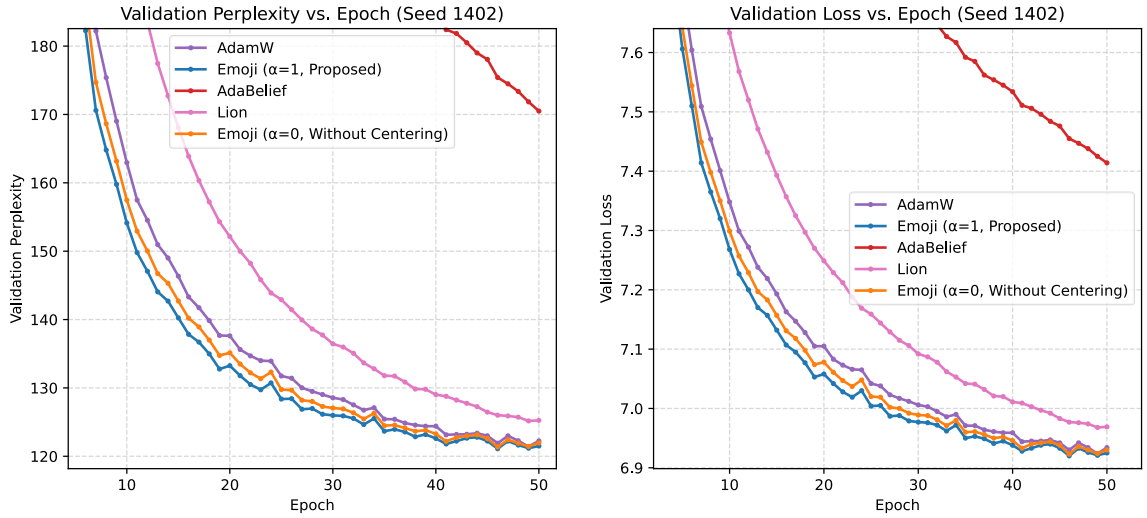


Figure S 2: Validation Perplexity (PPL) and Validation Loss versus Epoch for random seed 1402.

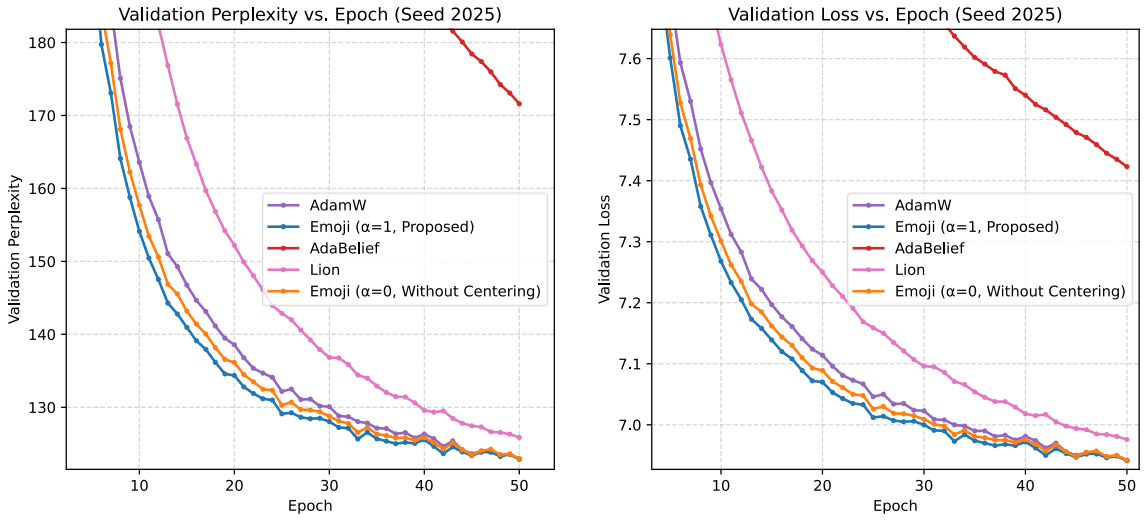
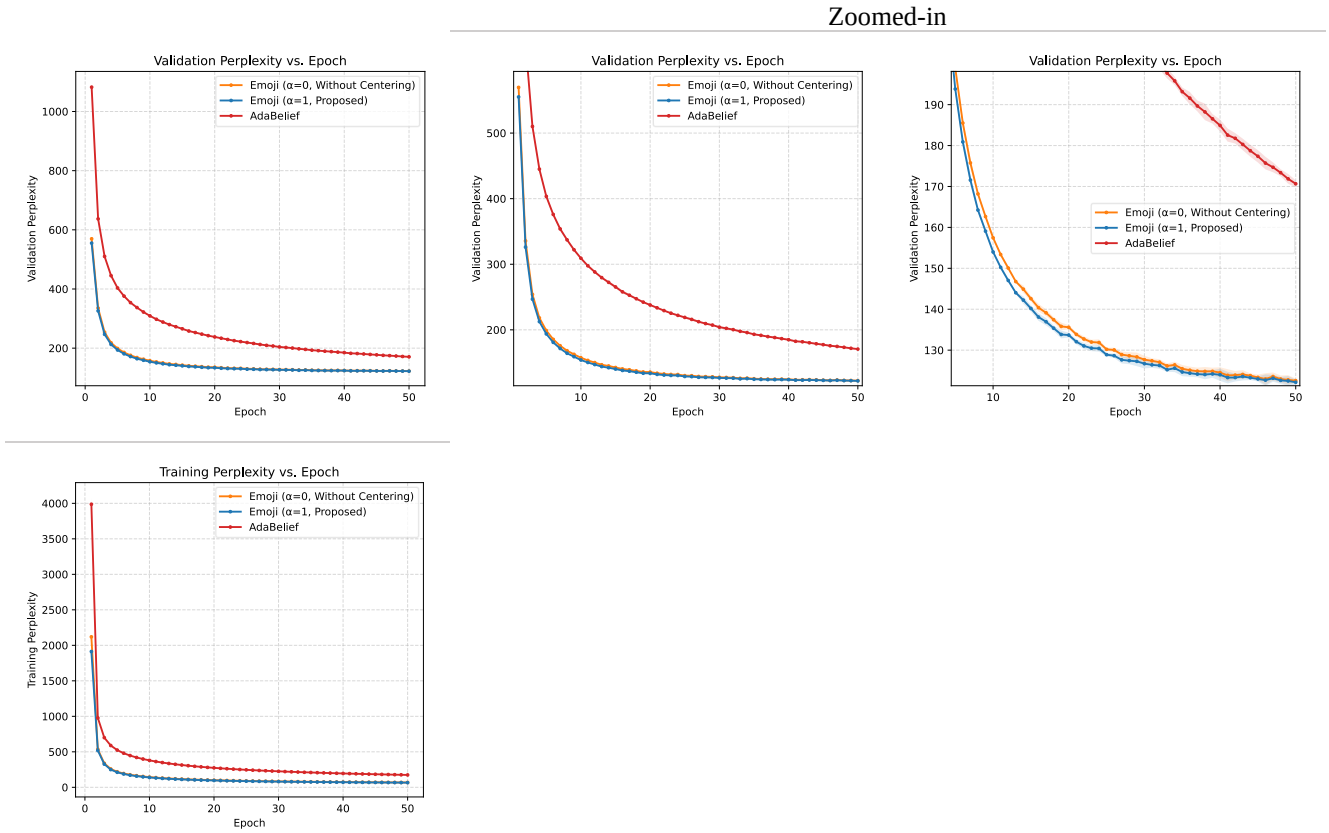


Figure S 3: Validation Perplexity (PPL) and Validation Loss versus Epoch for random seed 2025.



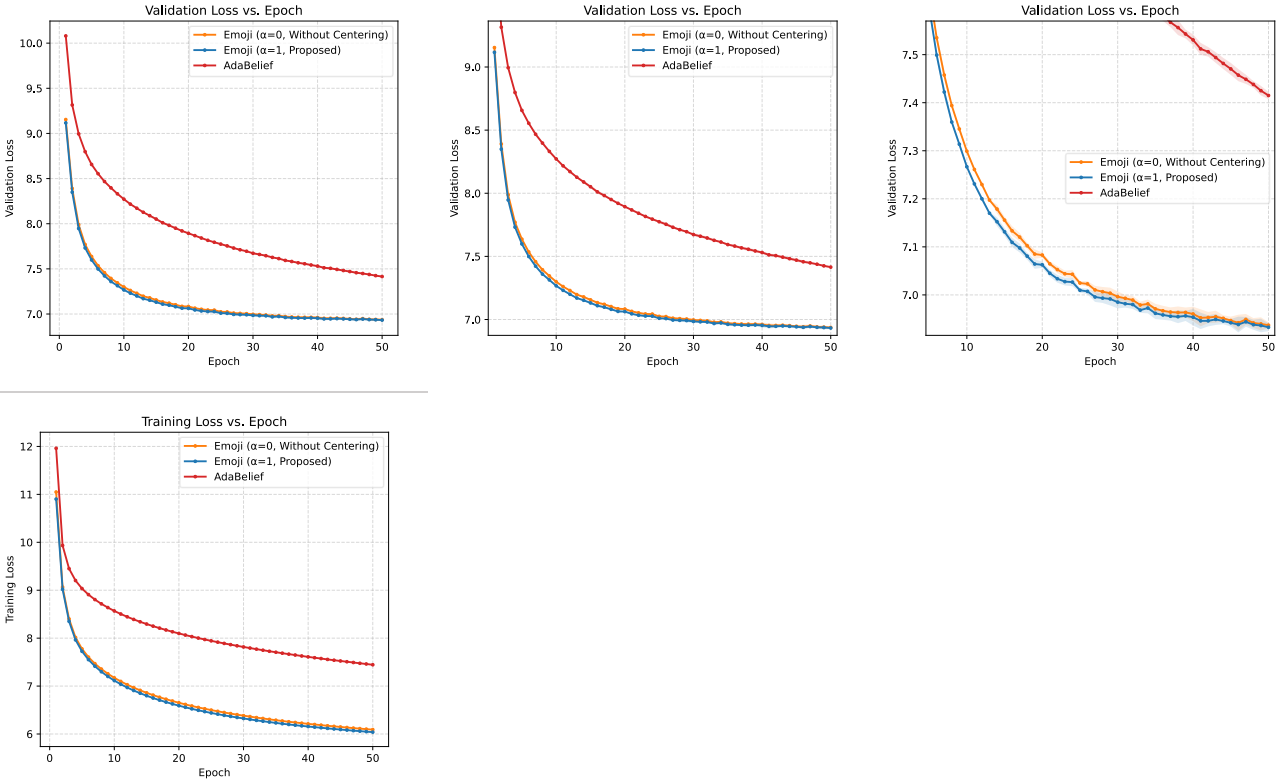
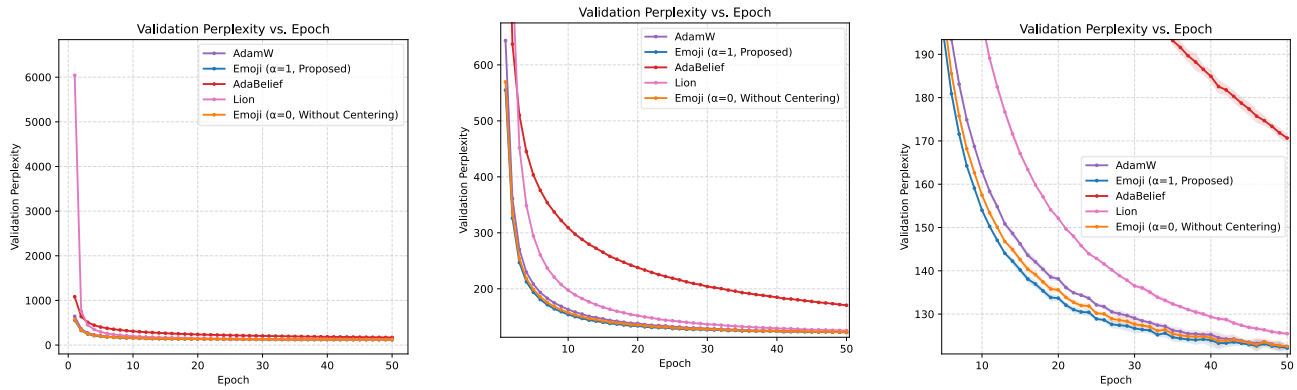


Figure S 4: Mean and standard deviation of the epoch-wise training and validation metrics for AdaBelief, EMOJI ($\alpha = 1$, proposed), and EMOJI ($\alpha = 0$, without centering) on WikiText-2 using a GPT-style Transformer. All configurations were evaluated using the same three random seeds. The shaded regions represent ±1 sample standard deviation across seeds.

Zoomed-in



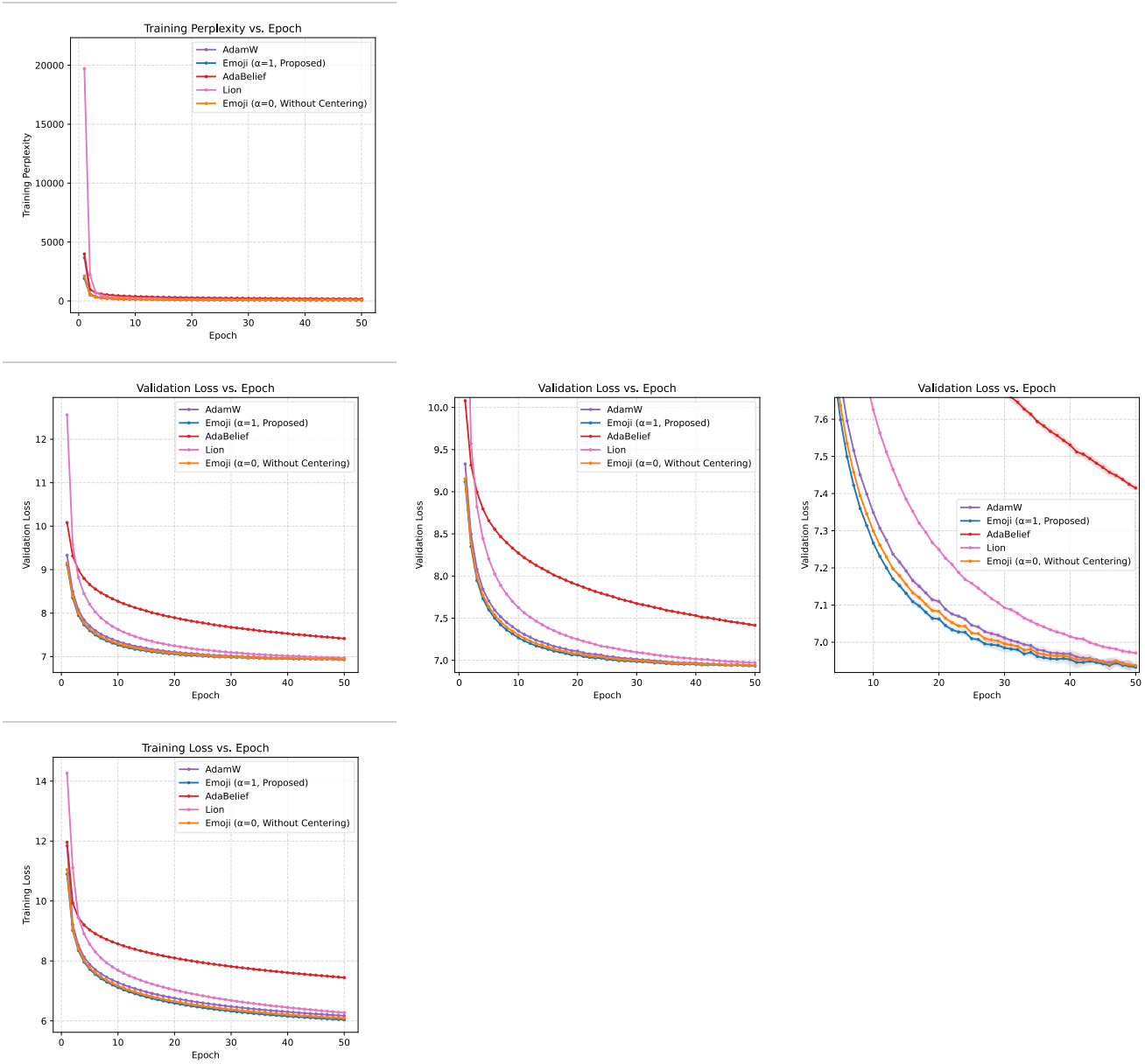
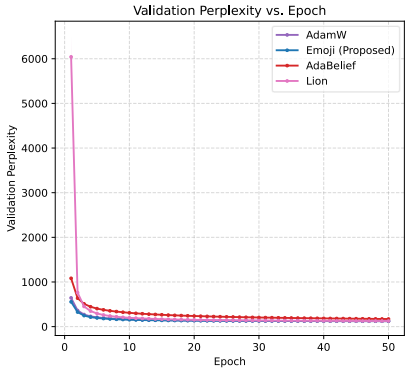
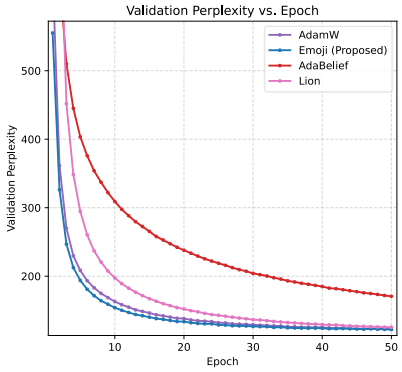
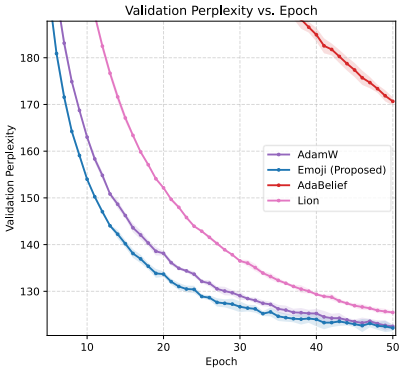


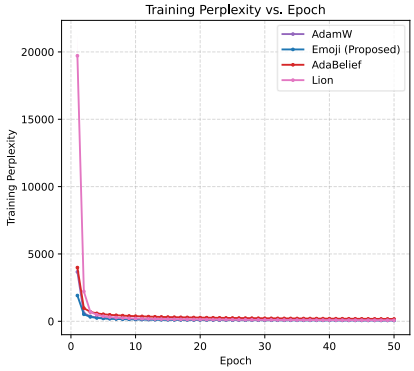
Figure S 5: Mean and standard deviation of the epoch-wise training and validation metrics for AdamW, AdaBelief, Lion, EMOJI ($\alpha = 1$, proposed), and EMOJI ($\alpha = 0$, without centering) on WikiText-2 using a GPT-style Transformer. All configurations were evaluated using the same three random seeds. The shaded regions represent ±1 sample standard deviation across seeds.

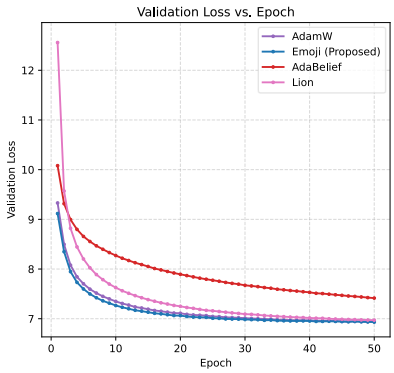
Zoomed-in

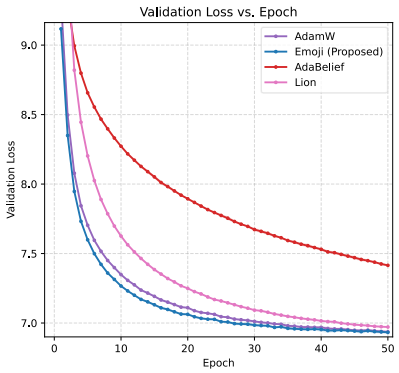

Validation Perplexity vs. Epoch
AdamW
Emoji (Proposed)
AdaBelief
Lion
Validation Perplexity
Epoch

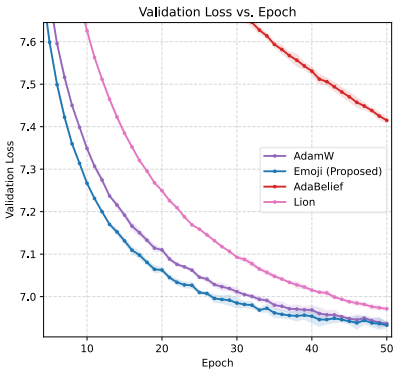

Validation Perplexity vs. Epoch
AdamW
Emoji (Proposed)
AdaBelief
Lion
Validation Perplexity
Epoch


Validation Perplexity vs. Epoch
AdamW
Emoji (Proposed)
AdaBelief
Lion
Validation Perplexity
Epoch


Training Perplexity vs. Epoch
AdamW
Emoji (Proposed)
AdaBelief
Lion
Training Perplexity
Epoch


Validation Loss vs. Epoch
AdamW
Emoji (Proposed)
AdaBelief
Lion
Validation Loss
Epoch


Validation Loss vs. Epoch
AdamW
Emoji (Proposed)
AdaBelief
Lion
Validation Loss
Epoch


Validation Loss vs. Epoch
AdamW
Emoji (Proposed)
AdaBelief
Lion
Validation Loss
Epoch

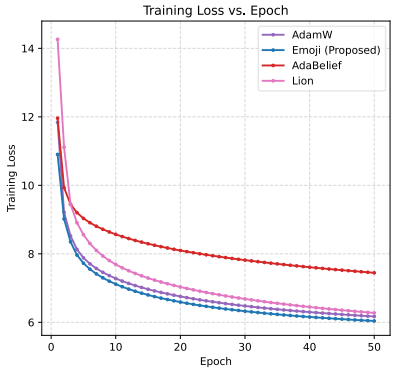


Figure S 6: Mean and standard deviation of the epoch-wise training and validation metrics for AdamW, AdaBelief, EMOJI, and Lion on WikiText-2 using a GPT-style Transformer. All optimizers were evaluated using the same three random seeds. The shaded regions represent ±1 sample standard deviation across seeds.

2.4. Transformer-Based Machine Translation on IWSLT14

We compared four optimizer implementations, AdamW, AdaBelief, EMOJI, and Lion, on the IWSLT14 German-to-English machine translation task using Fairseq. All runs used the `transformer_iwslt_de_en` architecture and the same dataset and shared training configuration. The experimental protocol used the same three random seeds across optimizer configurations.

2.4.1. Model Architecture and Training Configuration

The models used `Fairseq's transformer_iwslt_de_en` architecture. The saved configuration inspected for the EMOJI run confirmed six encoder layers and six decoder layers, 512-dimensional embeddings, feed-forward networks of dimension 1,024, and four attention heads in both the encoder and decoder. The model used ReLU activation, dropout of 0.3, zero attention and activation dropout, post-layer normalization, non-learned positional embeddings, and shared decoder input/output embeddings.

Table S 9: Model architecture and configuration.

Component	Configuration
Task	German-to-English machine translation
Dataset	IWSLT14 German-to-English
Fairseq task	translation
Fairseq architecture	transformer_iwslt_de_en
Encoder layers	6
Decoder layers	6
Encoder embedding dimension	512
Decoder embedding dimension	512
Encoder attention heads	4
Decoder attention heads	4
Attention head dimension	128
Encoder feed-forward dimension	1,024
Decoder feed-forward dimension	1,024

Activation function	ReLU
Decoder input/output embeddings	Shared
Encoder layer normalization	Post-LayerNorm (Pre-LN disabled)
Decoder layer normalization	Post-LayerNorm (Pre-LN disabled)
Embedding layer normalization	Disabled
Learned positional embeddings	Disabled
Maximum source positions	1,024 tokens
Maximum target positions	1,024 tokens
Model dropout	0.3
Attention dropout	0.0
Activation dropout	0.0

The shared training and evaluation configuration is summarized in Table S 10. The architecture details listed in Table S 9.

Table S 10: Shared training configuration.

Training parameter	**Value**
Maximum epochs	50
Maximum tokens per batch	4,096
Gradient accumulation (update_freq)	1 (Fairseq configuration)
Learning-rate scheduler	Inverse square root (inverse_sqrt)
Warm-up updates	4,000
Gradient clipping (clip_norm)	0.0 (disabled)
Training objective	Label-smoothed cross-entropy
Label-smoothing factor	0.1
Logging format	JSON
Logging interval	100 updates (stored checkpoint configuration)
Data-loader workers	1 (stored checkpoint configuration)
Validation frequency	Every epoch (stored checkpoint configuration)
Maximum validation tokens per batch	4,096 (stored checkpoint configuration)
Random seeds in the experiment protocol	1993, 1402, 2025

All command examples used the inverse-square-root learning-rate scheduler with 4,000 warm-up updates, disabled gradient clipping, dropout of 0.3, label-smoothed cross-entropy with a smoothing factor of 0.1, and a maximum of 4,096 tokens per batch. Optimizer-specific learning rates, beta coefficients, epsilon values, and weight decay are listed separately in Table S 11.

2.4.2. Optimizer Configurations

AdamW, AdaBelief, and EMOJI were evaluated using the same learning rate of 5×10^{-4}, weight decay of 0.0001, and moment coefficients $\beta_1 = 0.9$ and $\beta_2 = 0.98$. The choice of these coefficients follows the original Transformer paper, *Attention Is All You Need* [8], which used Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.98$. In our experiments, we retained these beta coefficients and used $\epsilon = 10^{-8}$, together with a common inverse-square-root learning-rate schedule with 4,000 warm-up updates. Applying the same settings to AdamW, AdaBelief, and EMOJI provides a controlled comparison of their optimization behavior under a shared configuration.

Table S 11: Optimizer configurations used for IWSLT14 German-to-English machine translation task using a Transformer architecture.

Optimizer	LR	β_1	β_2	WD Type	WD Value	ϵ
AdamW	5×10^{-4}	0.9	0.98	Decoupled	10^{-4}	10^{-8}
AdaBelief	5×10^{-4}	0.9	0.98	Decoupled	10^{-4}	10^{-8}
EMOJI (Proposed)	5×10^{-4}	0.9	0.98	Decoupled	10^{-4}	10^{-8}
Lion	5×10^{-5}	0.95	0.98	Decoupled	10^{-3}	——

Lion was configured according to recommendations in the Lion literature [7]. Its learning rate was set to one-tenth of the AdamW learning rate, while its weight decay was increased tenfold. This adjustment is motivated by Lion's sign-based update rule, which can produce updates with larger norms than AdamW's and therefore commonly calls for a smaller learning rate and a larger weight-decay coefficient. The coefficients $\beta_1 = 0.95$ and $\beta_2 = 0.98$ are also consistent with settings investigated for Lion in language-modeling experiments. These values are optimizer-specific choices adopted in this study rather than the default beta values of Lion [7].

The Lion configuration therefore follows the recommended relative scaling of learning rate and weight decay, rather than enforcing identical hyperparameters across all optimizers. Consequently, the comparison evaluates the four methods under their specified configurations, not under a strict requirement that every optimizer use identical optimizer-specific hyperparameters.

2.5. Fine-Tuning ALBERT-Base

To evaluate the proposed EMOJI optimizer across different aspects of natural language understanding, we conducted experiments on three GLUE tasks: CoLA, MRPC, and STS-B. We used the ALBERT-base-v2 model as implemented in the Hugging Face Transformers library [9], [10]. These tasks cover complementary forms of language understanding, including grammatical acceptability, paraphrase detection, and semantic textual similarity. CoLA is formulated as a single-sentence binary classification task that evaluates grammatical acceptability, whereas MRPC is a sentence-pair classification task that determines whether two sentences are semantically equivalent. STS-B evaluates semantic similarity between sentence pairs as a continuous regression task, with performance measured using Pearson and Spearman correlation coefficients. Together, these tasks provide a compact evaluation across distinct aspects of language understanding while remaining computationally manageable under limited hardware resources.

The ALBERT-base model was evaluated using four optimizers: AdamW, Lion, AdaBelief, and the proposed EMOJI optimizer. Each optimizer was evaluated using five independent random seeds. The same set of seeds was used across all four optimizers to ensure a consistent basis for comparison and to reduce the influence of stochastic variation. The mean performance across the five runs was used as the primary evaluation criterion.

The general ALBERT-base fine-tuning configuration was adopted from the original ALBERT configuration, including task-specific learning rates, dropout rates, training steps, warmup steps, and maximum sequence length. Because of GPU memory constraints, the batch size was set to 8 for all three tasks (CoLA, MRPC, and STS-B). The remaining task-level training settings were kept unchanged. For all four optimizers the ALBERT dropout rate was set to 0.0, the classifier dropout rate to 0.1, and the maximum sequence length to 512. The number of training steps and warmup steps was kept consistent with the corresponding ALBERT configuration for each task.

Optimizer-specific hyperparameters were configured according to the corresponding reference settings:

AdamW and AdaBelief used $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$, EMOJI also used the same β_1, β_2, and ϵ values, and Lion used $\beta_1 = 0.9$ and $\beta_2 = 0.99$. Lion was configured according to the recommendations established in the original Lion literature and followed consistently in all previous experiments. Because Lion uses a sign-based update, the magnitude of its parameter updates tends to be larger than that of Adam-style optimizers [7]. Consequently:

The learning rate for Lion was set to one order of magnitude smaller than the corresponding Adam-family learning rate (e.g., 10^{-6} vs 10^{-5} on CoLA, 2×10^{-6} vs 2×10^{-5} on MRPC and STS-B). The weight-decay

coefficient was increased (0.1 instead of 0.01) to compensate for the larger update norms and to maintain training stability.

These relative scalings of learning rate and weight decay are optimizer-specific recommendations rather than a requirement that every optimizer share identical hyper-parameters. All other training settings (architecture, dropout, number of steps, warmup, maximum sequence length, etc.) remained identical across the four optimizers.

Table S 12: Hyperparameter configurations used for AdamW, Lion, AdaBelief, and EMOJI on the CoLA task using ALBERT-base.[1]

Optimizer	LR	BSZ	ALBERT DR	Classifier DR	TS	WS	MSL	WD	β_1	β_2	ϵ
AdamW	10^{-5}	8	0.0	0.1	5336	320	512	0.01	0.9	0.999	10^{-8}
Lion	10^{-6}	8	0.0	0.1	5336	320	512	0.1	0.9	0.99	—
AdaBelief	10^{-5}	8	0.0	0.1	5336	320	512	0.01	0.9	0.999	10^{-8}
EMOJI (Proposed)	10^{-5}	8	0.0	0.1	5336	320	512	0.01	0.9	0.999	10^{-8}

Table S 13: Hyperparameter configurations used for AdamW, Lion, AdaBelief, and EMOJI on the MRPC task using ALBERT-base.

Optimizer	LR	BSZ	ALBERT DR	Classifier DR	TS	WS	MSL	WD	β_1	β_2	ϵ
AdamW	2×10^{-5}	8	0.0	0.1	800	200	512	0.01	0.9	0.999	10^{-8}
Lion	2×10^{-6}	8	0.0	0.1	800	200	512	0.1	0.9	0.99	—
AdaBelief	2×10^{-5}	8	0.0	0.1	800	200	512	0.01	0.9	0.999	10^{-8}
EMOJI (Proposed)	2×10^{-5}	8	0.0	0.1	800	200	512	0.01	0.9	0.999	10^{-8}

Table S 14: Hyperparameter configurations used for AdamW, Lion, AdaBelief, and EMOJI on the STS-B task using ALBERT-base.

Optimizer	LR	BSZ	ALBERT DR	Classifier DR	TS	WS	MSL	WD	β_1	β_2	ϵ
AdamW	2×10^{-5}	8	0.0	0.1	3598	214	512	0.01	0.9	0.999	10^{-8}
Lion	2×10^{-6}	8	0.0	0.1	3598	214	512	0.1	0.9	0.99	—
AdaBelief	2×10^{-5}	8	0.0	0.1	3598	214	512	0.01	0.9	0.999	10^{-8}
EMOJI (Proposed)	2×10^{-5}	8	0.0	0.1	3598	214	512	0.01	0.9	0.999	10^{-8}

2.5.1. Model Configuration (ALBERT-base-v2)

The following table summarizes the architecture and key settings of the ALBERT-base-v2 model used in all GLUE experiments (CoLA, MRPC, and STS-B). The model follows the standard ALBERT-base configuration with parameter sharing across layers and a factorized embedding parameterization.

Table S 15: Model Configuration (ALBERT-base-v2)

Setting	Value
Architecture	ALBERT-base-v2
Hidden size	768
Number of attention heads	12
Number of hidden layers	12
Intermediate size	3072
Embedding size	128
Vocabulary size	30 000
Maximum position embeddings	512
Classifier dropout	0.1
Attention / hidden dropout	0.0

[1] **Abbreviations:** LR, learning rate; BSZ, batch size; ALBERT DR, ALBERT dropout rate; Classifier DR, classifier dropout rate; TS, training steps; WS, warmup steps; MSL, maximum sequence length; WD, weight decay.

These settings were kept identical across all optimizers and all three tasks to ensure a controlled comparison.

2.6. Image Classification on ImageNet-1K with ResNet-18

To further evaluate the scalability of the proposed EMOJI optimizer on a larger-scale image classification task, we conducted experiments on ImageNet-1K using the ResNet-18 architecture for 100 epochs, with a batch size of 256. Due to the substantially higher computational cost of ImageNet-1K training, the evaluation was limited to the proposed EMOJI optimizer and AdaBelief, and each optimizer was evaluated in a single training run under the same experimental configuration and using the same random seed. Accordingly, the results in this section are presented as a direct comparison between the two runs rather than as estimates aggregated across multiple independent seeds. Rather than performing an extensive hyperparameter search for this dataset, the optimizer configurations were selected based on standard values reported in the literature and, in particular, the AdaBelief reference configuration for the same architecture and dataset.

For both AdaBelief and EMOJI, the learning rate, β_1, β_2, and weight decay were configured according to the values summarized in Table S 16. The numerical-stability constant was set to $\epsilon = 10^{-8}$ for both optimizers, consistent with the AdaBelief configuration reported for ResNet-18 on ImageNet-1K.

A step-wise learning-rate schedule was applied during training, with the learning rate reduced by a factor of 10 at epochs 30, 60, and 90. This schedule allows the optimizers to proceed with progressively smaller updates during the intermediate and later stages of training as optimization approaches lower-loss regions. The corresponding optimizer configurations are summarized in Table S 16.

Table S 16: Optimizer hyperparameter configurations used for ResNet-18 on ImageNet-1K.

Optimizer	**LR**	$\boldsymbol{\beta_1}$	$\boldsymbol{\beta_2}$	**WD Type**	**WD Value**	$\boldsymbol{\epsilon}$
EMOJI (Proposed)	0.001	0.9	0.999	Decoupled	0.01	10^{-8}
AdaBelief	0.001	0.9	0.999	Decoupled	0.01	10^{-8}

Table S 17: Training configuration and experimental settings for ResNet-18 on ImageNet-1K.

Setting	**Value**
Architecture	ResNet-18
Dataset	ImageNet-1K
Number of epochs	100
Batch size	256
Learning-rate schedule	Step decay (×0.1) at epochs 30, 60, and 90
Random seed	42
Weight decay type	Decoupled
Warmup	Disabled

2.6.1. Model Architecture (ResNet-18)

The experiments use the standard ResNet-18 architecture as implemented in `torchvision.models.resnet18`. The network consists of an initial 7×7 convolution followed by four residual stages with BasicBlocks and a final fully-connected classification layer.

Table S 18: Model architecture specifications of ResNet-18 used in the ImageNet-1K experiments.

Setting	Value
Architecture	ResNet-18
Input channels	3 (RGB)
Stem convolution	7×7, 64 channels
Block type	BasicBlock
Stage configuration	[2, 2, 2, 2]
Channel progression	64 → 128 → 256 → 512
Number of classes	1000
Normalization	BatchNorm2d
Activation	ReLU
Approximate number of parameters	11.7 million

All residual stages use the standard BasicBlock design with two 3×3 convolutions. Downsampling is performed by 1×1 convolutions in the first block of stages 2, 3, and 4. The final feature map is globally average-pooled and passed through a linear layer that produces logits for the 1000 ImageNet classes.

Reference

[1] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in International Conference on Learning Representations (ICLR), 2019.

[2] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the Importance of Initialization and Momentum in Deep Learning," in *Proceedings of the 30th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 28. PMLR, 2013, pp. 1139–1147.

[3] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in International Conference on Learning Representations (ICLR), 2015.

[4] J. Zhuang et al., "AdaBelief Optimizer: Adapting Stepsizes by the Belief in Observed Gradients," in Advances in Neural Information Processing Systems, Curran Associates, Inc., 2020, pp. 18795–18806.

[5] M. Zaheer, S. Reddi, D. Sachan, S. Kale, and S. Kumar, "Adaptive Methods for Nonconvex Optimization," in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2018.

[6] L. Luo, Y. Xiong, Y. Liu, and X. Sun, "Adaptive Gradient Methods with Dynamic Bound of Learning Rate," in International Conference on Learning Representations (ICLR), 2019.

[7] X. Chen *et al.*, "Symbolic discovery of optimization algorithms," *Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 49205–49233, 2023.

[8] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.

[9] T. Wolf *et al.*, "Transformers: State-of-the-Art Natural Language Processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45.

[10] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020.